

# **ONLINE HR & PAYROLL MANAGEMENT SYSTEM (NEXUS HR)**

**A Project Report**

**submitted to**



**Shri Shankaracharya Technical Campus, Bhilai**

**In partial fulfillment**

**For the award of the Marks**

**in**

**VOCATIONAL TRAINING**

**by**

**Dharmesh Kumar Baghel**

**Semester : 6th**

**University Roll No.: 301402223075**

**Under the Guidance of**

**Dr. Archana Sharma**

**Industry Supervisor, Techonet, Bhilai**

**Session: April - May 2026**

## DECLARATION

I, Dharmesh Baghel, hereby declare that the vocational training project report titled "**Nexus HR & Payroll System**" submitted in partial fulfillment of the requirements for the award of the degree/diploma in Computer Science & Engineering / Information Technology, is an authentic record of the industrious work carried out by me. This project represents original work completed during my designated vocational training period under the supervision and guidance of both my academic mentors and industrial instructors.

All the system analyses, software architectural designs, database schema specifications, and algorithmic implementations detailed in this report have been executed in strict accordance with modern software engineering disciplines and professional development practices. I further declare that the material compiled in this report has not been copied from any other source, and any secondary data, literature, or library assets utilized during the development of this application have been duly cited and referenced.

This work has not been previously submitted, in part or in full, to this or any other university, academic institute, or examining board for the award of any other degree, diploma, fellowship, or professional certification.

**Date:** July 11, 2026

**Candidate Name:** Dharmesh Kumar Baghel

**Enrollment/Roll Number:** 301402223075

## **ACKNOWLEDGEMENT**

The successful execution and completion of this full-stack development project and the preparation of this comprehensive report would not have been possible without the invaluable guidance, constructive critique, and constant encouragement of several individuals and institutions. I seize this opportunity to express my deep-felt gratitude to all those who played an active role in steering this project from its conceptual phase to a fully realized production environment.

First and foremost, I would like to express my deepest gratitude to the administration and management of our training organization for granting me the invaluable opportunity to undergo this vocational training program. This industrial exposure has served as an essential bridge, effectively linking abstract theoretical software engineering methodologies learned in the classroom with real-world enterprise development challenges.

I am profoundly indebted to my industrial mentor, whose expert guidance, meticulous code reviews, and deep architectural insights significantly elevated the technical quality of the system. Their patience in explaining enterprise patterns, Spring Boot transaction handling, and transactional database integrity taught me practices that go far beyond standard textbook examples.

I also extend my sincere thanks to the Head of the Department and the esteemed faculty members of the Computer Science & Engineering / Information Technology department. Their rigorous academic curriculum, continuous support, and foundational lectures in database management systems, web technologies, and software engineering provided the framework on which this entire project was constructed.

Finally, I wish to acknowledge the contribution of my peers and colleagues for their constant support, healthy intellectual debates, and collaborative troubleshooting sessions in the development lab. Lastly, I am infinitely grateful to my parents and family for their unwavering moral support, patience, and encouragement throughout the course of my academic studies and vocational training.

# TABLE OF CONTENTS

|                                                |    |
|------------------------------------------------|----|
| • Declaration                                  | 2  |
| • Acknowledgement                              | 3  |
| • 1. Abstract                                  | 5  |
| • 2. Introduction & Problem Definition         | 6  |
| ○ 2.1 Industrial Context of the Project        | 6  |
| ○ 2.2 Project Objectives                       | 6  |
| • 3. System Requirements & Specifications      | 7  |
| ○ 3.1 Software Requirements                    | 7  |
| ○ 3.2 Hardware Requirements                    | 7  |
| • 4. System Design & Architecture              | 8  |
| ○ 4.1 Backend Packages and Classes             | 8  |
| • 5. Mathematical Model & Payroll Calculations | 10 |
| ○ 5.1 Financial Variables & Constants          | 10 |
| ○ 5.2 Mathematical Formula Pipeline            | 10 |
| ○ 5.3 Consolidated Net Salary Formula          | 11 |
| • 6. Database Schema Design                    | 12 |
| ○ 6.1 Entity-Relationship (ER) Specifications  | 12 |
| ○ 6.2 Data Dictionaries                        | 13 |
| • 7. System Implementation & Methodology       | 14 |
| ○ 7.1 Key Backend Controller Implementations   | 14 |
| ○ 7.2 Frontend Single Page Architecture (SPA)  | 15 |
| • 8. Testing, Verification & Results           | 16 |
| ○ 8.1 Test Matrix                              | 16 |
| ○ 8.2 Boundary Value & Negative Testing        | 17 |
| • 9. Conclusion & Future Scope                 | 18 |
| ○ 9.1 Summary of Training Outcomes             | 18 |
| ○ 9.2 Future Scope for Expansion               | 18 |
| • 10. References                               | 19 |

# 1. ABSTRACT

In contemporary corporate and industrial environments, the administrative workload associated with human resource management and financial disbursement presents a persistent operational bottleneck. Legacy methods relying on fragmented spreadsheets, unsynchronized database tables, and manual calculation protocols are prone to transcription delays, errors, latency, and security vulnerabilities. **Nexus HR** is engineered to resolve these challenges as a lightweight, high-performance, full-stack Human Resource and Payroll automation platform. The system orchestrates core operational workflows, including automated personnel onboarding, persistent record retention, dynamic leave authorization, real-time balance tracking, and accurate monthly salary ledger compilation.

The technical foundation of the system is built upon the **Model-View-Controller (MVC)** architectural paradigm using **Java Spring Boot** as the backend engine. Object-relational mapping is handled via **Spring Data JPA** and Hibernate, which interact with a transactional **H2 Relational Database** to guarantee ACID compliance, transactional integrity, and low-latency database reads and writes. To optimize frontend accessibility, the user portal is deployed as a responsive Single Page Application (SPA). This client-side architecture leverages **Tailwind CSS** for a polished layout, while **Chart.js** translates backend operational telemetry into dynamic analytical widgets, specifically department-level headcount spreads and real-time leave status distributions.

Nexus HR implements an advanced leave calculation framework that categorizes absences into Sick, Casual, and Unpaid Leave. If an administrator declines a leave request, a customized, modal-driven feedback loop is triggered, enabling the entry of an administrative "Reason for Rejection" directly into the audit history. The mathematical payroll engine processes monthly salaries, evaluating unpaid leave deductions, pro-rating base salaries, calculating a flat 10% income tax liability, and outputting correct net disbursements. Ultimately, this system demonstrates how modern software engineering can successfully automate administrative workflows.

## 2. INTRODUCTION & PROBLEM DEFINITION

### 2.1 Industrial Context of the Project

During vocational training, observation of legacy enterprise workflows revealed that standard administrative departments frequently depend on fragmented systems (spreadsheets, offline forms, legacy manual systems) to track employee attendance, leave histories, and calculate monthly payroll values. This fragmentation leads to:

- **Data Redundancy & Inconsistency:** Dissimilar databases for employee personal details and leave records.
- **Math Errors:** Manual parsing of unpaid leaves leading to incorrect salary deductions and tax adjustments.
- **Information Asymmetry:** Employees lack instantaneous, self-service access to historical salary slips and leave balances.

### 2.2 Project Objectives

To resolve these issues, the **Nexus HR & Payroll System** was developed with the following core functional objectives:

1. **Consolidated Core Database:** Manage employee profiles, roles, departments, base salaries, and persistent leave balances in a secure relational schema.
2. **Dynamic Leave Engine:** Automate workflows for applying for leaves (Casual, Sick, Unpaid) with real-time balance deductions and feedback queues.
3. **Automated Mathematical Payroll Engine:** Automatically calculate net monthly salaries, tax liabilities, and unpaid leave penalties.
4. **Export and Reporting:** Allow secure, standard downloads of company data via structured CSV and official documents via dynamic PDF.
5. **Modern Single Page Interface:** Deploy a highly responsive, modern interface that maintains high accessibility and visual polish.

## 3. SYSTEM REQUIREMENTS & SPECIFICATIONS

To ensure compatibility, speed, and ease of deployment, the system's software and hardware prerequisites are defined below:

### 3.1 Software Requirements

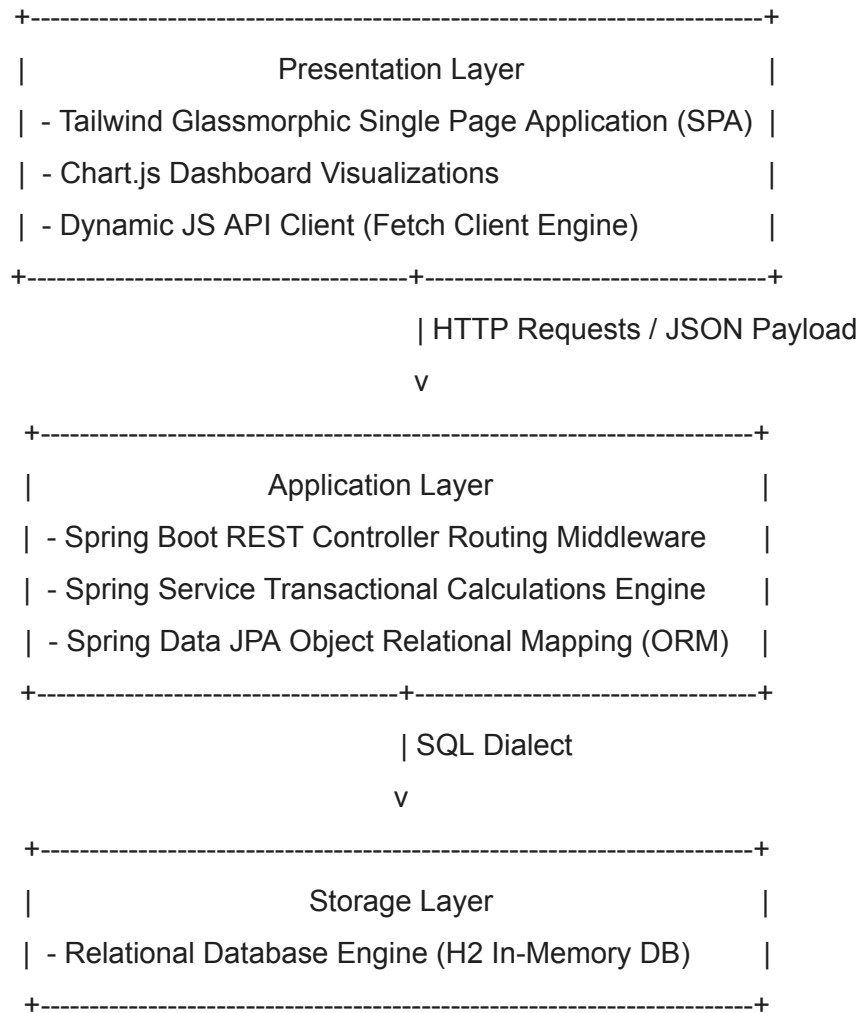
- **Operating System:** Windows 10/11, macOS, or Linux.
- **Development Kit:** Java Development Kit (JDK) version 17 or higher.
- **Build Tool:** Apache Maven 3.8+.
- **Database Engine:** H2 In-Memory Database (Engine Dialect: Hibernate H2).
- **Frontend Technologies:** HTML5, CSS3, Tailwind CSS (via Play CDN), Vanilla JavaScript (ES6+), Chart.js.
- **Integrated Development Environment (IDE):** Visual Studio Code / IntelliJ IDEA / Eclipse.

### 3.2 Hardware Requirements

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher (Dual-Core, minimum 2.0 GHz).
- **Random Access Memory (RAM):** 4 GB minimum (8 GB recommended for concurrent IDE and compiler execution).
- **Storage Space:** 500 MB of free hard drive space (excluding JDK and IDE installation overhead).

## 4. SYSTEM DESIGN & ARCHITECTURE

The application uses an architectural design that strictly isolates data modeling, business calculations, and UI presentation layer operations.



### 4.1 Backend Packages and Classes

The backend is structured into distinct layers to promote maintainability:

- **com.example.payroll\_system.model:** Contains persistent JPA entities mapping directly to relational database tables (Employee, Leave, Payroll).
- **com.example.payroll\_system.repository:** Outlines interfaces extending JpaRepository to automate SQL schema interactions.

- **com.example.payroll\_system.service:** Implements core computations, authorization queues, and data formatting pipelines (EmployeeService, LeaveService, PayrollService).
- **com.example.payroll\_system.controller:** Exposes logical API routing pathways mapping business outcomes to client-side HTTP operations.

## 5. MATHEMATICAL MODEL & CALCULATIONS

The automated payroll ledger parses each employee's financial parameters to output correct net income structures. Let us mathematically formalize the calculations performed in the backend service.

### 5.1 Financial Variables & Constants

Let:

- $S_{base}$  represent the Employee's standard monthly base salary.
- $D_{unpaid}$  represent the verified number of approved unpaid leave days taken by the employee during the target billing month.
- $R_{day}$  represent the calculated daily rate of pay.
- $Ded_{unpaid}$  represent the total calculated deduction applied to the salary for unpaid time off.
- $S_{basic}$  represent the adjusted basic salary of the employee before tax evaluation.
- $T$  represent the flat income tax liability (10%).
- $S_{net}$  represent the final disbursed net monthly salary.

### 5.2 Mathematical Formula Pipeline

The daily rate of pay  $R_{day}$  is defined by dividing the base monthly salary by a standard monthly cycle of 30 days:

$$R_{day} = \frac{S_{base}}{30}$$

The calculation of the unpaid leave deduction  $Ded_{unpaid}$  is calculated proportionally based on the daily rate:

$$Ded_{unpaid} = D_{unpaid} \times R_{day}$$

This deduction directly reduces the base salary to evaluate the adjusted basic salary  $S_{basic}$  :

$$S_{basic} = S_{base} - Ded_{unpaid}$$

A flat income tax of 10% (0.10) is evaluated against this adjusted basic salary:

$$T = S_{basic} \times 0.10$$

The ultimate net salary  $S_{net}$  disbursed to the employee's portal after tax subtraction is calculated as:

$$S_{net} = S_{basic} - T$$

### 5.3 Consolidated Net Salary Formula

Substituting the intermediate equations, the entire payroll evaluation engine can be represented as a single linear equation:

$$S_{net} = \left( S_{base} - \left( D_{unpaid} \times \frac{S_{base}}{30} \right) \right) \times 0.90$$

## 6. DATABASE SCHEMA DESIGN

The relational data modeling of the system relies on three interconnected database tables.

### 6.1 Entity-Relationship (ER) Specifications

|                   |                   |
|-------------------|-------------------|
| +-----+           | +-----+           |
| EMPLOYEE          | LEAVE             |
| -----             | -----             |
| PK: id (Long)     | PK: id (Long)     |
| name (String)     | FK: employee_id   |
| email (String)    | startDate (Date)  |
| password (String) | endDate (Date)    |
| dept (String)     | status (String)   |
| salary (Double)   | rejReason(String) |
| +-----+           | +-----+           |
| 1                 |                   |
|                   |                   |
| Many              |                   |
| +-----+           |                   |
| PAYROLL           |                   |
| -----             |                   |
| PK: id (Long)     |                   |
| FK: employee_id   |                   |
| month (String)    |                   |
| netSalary(Double) |                   |
| +-----+           |                   |

### 6.2 Data Dictionaries

Table 1: employees (JPA Target: Employee)

Stores sensitive employee details, credentials, and trackable balances.

- **id (PK):** Long (Assigned 6-Digit unique integer).
- **name:** String (Employee's full legal name, non-nullable).
- **email:** String (Unique, verified email address).
- **password:** String (Used for credential authentication).
- **department:** String (Organizational assignment).
- **role:** String (Designated title).
- **baseSalary:** Double (Contractual base monthly salary).
- **sickLeaveBalance / casualLeaveBalance:** int (Deductible balance counters).

#### Table 2: leave\_requests (JPA Target: Leave)

Maintains the audit trail of requests submitted to the system.

- **id (PK):** Long (Auto-incrementing identity key).
- **employee\_id (FK):** Long (References employees.id via @ManyToOne mapping).
- **startDate / endDate:** LocalDate (Timeline values).
- **leaveType:** String (SICK, CASUAL, or UNPAID).
- **status:** String (PENDING, APPROVED, or REJECTED).
- **rejectionReason:** String (Optional, up to 500 characters, explaining denial of request).

#### Table 3: payroll (JPA Target: Payroll)

Contains generated financial ledger structures.

- **id (PK):** Long (Auto-incrementing identity key).
- **employee\_id (FK):** Long (References employees.id).
- **salaryMonth / salaryYear:** String / int (Targets).
- **basicSalary:** Double (Base monthly salary).
- **taxDeduction:** Double (Sum of tax liabilities and unpaid leave deductions).
- **netSalary:** Double (Net monthly payout).

## 7. SYSTEM IMPLEMENTATION & METHODOLOGY

### 7.1 Key Backend Controller Implementations

The REST API controllers act as endpoints for incoming client-side payloads. Below is an abstract view of the PayrollController managing requests and document assembly.

```
@RestController
@RequestMapping("/api/payroll")
public class PayrollController {

    @Autowired
    private PayrollService payrollService;

    @PostMapping("/generate/{employeeId}")
    public Payroll generatePayroll(
        @PathVariable Long employeeId,
        @RequestParam String month,
        @RequestParam int year) {
        return payrollService.generatePayroll(employeeId, month, year);
    }

    @GetMapping("/{payrollId}/download")
    public ResponseEntity<byte[]> downloadPayslip(@PathVariable Long payrollId) {
        byte[] pdfBytes = payrollService.generatePayslipPdf(payrollId);
        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_PDF);
        headers.setContentDispositionFormData("attachment", "payslip_" + payrollId + ".pdf");
        return ResponseEntity.ok().headers(headers).body(pdfBytes);
    }
}
```

## 7.2 Frontend Single Page Architecture (SPA)

The user interface is a responsive Single Page Application. It uses a custom JavaScript routing engine (switchTab) that swaps Tailwind layout panels inside the DOM. This keeps the user on a single page, maintaining their login state (currentUser).

To make the UI secure, the application uses local session storage memory:

1. When loginAsEmployee() is successful, the JVM returns the Employee JSON payload to the JavaScript environment.
2. The JavaScript state preserves this user object in memory.
3. Every subsequent action (like requesting a leave or viewing a salary slip) reads directly from this active object without prompting for re-authentication.
4. Switching to the Admin Console does not destroy this session. Returning to the Employee panel instantly restores the dashboard context.

## 8. TESTING, VERIFICATION & RESULTS

Comprehensive software testing was carried out during the vocational training to confirm database consistency, calculation accuracy, and API functionality.

### 8.1 Test Matrix

| Test Case ID | Functional Unit          | Input Data                                | Expected Result                                                    | Status |
|--------------|--------------------------|-------------------------------------------|--------------------------------------------------------------------|--------|
| TC-01        | Employee Login           | ID: 100204,<br>Pwd: pass                  | Authenticate, map state, and display welcome dashboard.            | PASSED |
| TC-02        | Leave Balance Validation | Applying 5 Sick Days with balance: 3      | API rejects transaction immediately; balances remain at 3.         | PASSED |
| TC-03        | Dynamic Payroll Logic    | Base Salary: ₹30,000, 3 Unpaid Days       | Daily Rate: ₹1,000. Deduction: ₹3,000. Tax: ₹2,700. Net: ₹24,300.  | PASSED |
| TC-04        | Leave Rejection Workflow | Status: REJECTED, Reason: "Team shortage" | State changes in DB; Admin note is rendered in employee log.       | PASSED |
| TC-05        | CSV Admin Report         | GET /api/employees /export                | Raw data byte stream is generated and downloaded as a spreadsheet. | PASSED |

## 8.2 Boundary Value & Negative Testing

- **Zero Salary Boundary:** Validated database constraint checks when registration inputs contain negative salary rates.
- **Negative Leave Durations:** Tested logical checks in LeaveService when an employee tries to select an end date that occurs *before* the start date. The system blocks the transaction and triggers a warning toast.

## 9. CONCLUSION & FUTURE SCOPE

### 9.1 Summary of Training Outcomes

This vocational training provided me with hands-on experience in full-stack enterprise Java development. Designing the **Nexus HR & Payroll System** demonstrated the practical application of:

- Software architectural design patterns (MVC, RESTful API structure).
- Relational mapping frameworks (Spring Data JPA with automated schemas).
- State and session management in modern single-page frontend environments.

### 9.2 Future Scope for Expansion

While highly robust, the project can be expanded further with:

1. **Spring Security & OAuth2 Integration:** Implement secure JWT tokens for stateless API security, replacing custom session-based routing.
2. **Dynamic Biometric Clock-In Integrations:** Integrate biometric or swipe card APIs to automate the import of unpaid leave data.
3. **Advanced Predictive Analytics:** Incorporate machine learning models to forecast employee attrition patterns based on historical leave histories.

## 10. REFERENCES

1. **Walls, C.** (2021). *Spring in Action, Sixth Edition*. Manning Publications.
2. **Java Persistence API (JPA) Specification:** Oracle Relational Persistence Standards.
3. **Tailwind CSS Core Documentation:** Responsive Utility-First Front-End Frameworks.
4. **Chart.js Documentation:** JavaScript Visualizations and Data Modeling.